

PRODUCT & TECHNICAL DOSSIER

Technical Exercise Submission

Comprehensive coverage of product, system design, quality, security, and future plans.

CONTENTS

1. Product and Experience

- 1.1 Purpose
- 1.2 Problem
- 1.3 Users
- 1.4 User Stories
- 1.5 Journeys
- 1.6 Requirements
- 1.7 Scope
- 1.8 Key Experience Decisions

2. System Design

- 2.1 Explanations & Architecture Narrative
- 2.2 Architecture Diagram
- 2.3 Context Diagram
- 2.4 Component Diagram & Solution Structure
- 2.5 Deployment Diagram
- 2.6 Database Schemas
- 2.7 Data Model
- 2.8 Data Flows
- 2.9 APIs & Contracts
- 2.10 Integrations & Dependencies
- 2.11 Authentication & Access
- 2.12 Infrastructure & Environments

3. Quality and Security

- 3.1 Testing
- 3.2 Observability
- 3.3 Security Model
- 3.4 Threat & Risk Analysis
- 3.5 Data & Privacy Decisions
- 3.6 Known Limitations

4. Future and Production

- 4.1 Roadmap
- 4.2 Product & Technical Vision
- 4.3 Remaining Work
- 4.4 Time, Effort, People & Roles
- 4.5 Costs
- 4.6 Dependencies
- 4.7 Assumptions

1. PRODUCT AND EXPERIENCE

1.1 PURPOSE

The purpose of this product is to provide a simplified, authenticated API that serves up-to-date product information — and an admin portal that makes those updates easy for non-technical staff — so that further applications and clients can leverage the data without requiring a repository deploy every time content changes.

Specifically, this enables further development of applications such as the WhatsApp Course Registration service (in particular, the Content Distribution implementation) and the Alpha Guest App. It could also serve as the starting point for an overhauled, modernised Courses service for active courses, linked to current Domain Services.

For this POC, WhatsApp Content Distribution is motivation, not a drop-in spec and not something measured against. What is built is a concept catalog API (all product data stored) plus an admin portal. If the service later looks viable, a consuming app can be adapted then. The live demo of the read API is a wwwroot page on ApplicationRoot — buttons in the same family as other Alpha BFFs and the IDP demo page.

Product Definition

A product is one content edition of an Alpha course: a series of video episodes (plus optional training videos) with its own copy. Product meta includes at least title and description. Episode meta includes at least a title, plus a video hosted on Brightcove (Vimeo remains a provider value; Brightcove is the pattern this catalog follows). Live host APIs stay out of this POC.

Family is a named line of courses (Alpha Film Series, Alpha Youth Series, Marriage Course, ...). It is a linked property on a product, not a parent that owns nested editions.

Different video and audio content = a new product. Alpha Film Series and Alpha Film Series Africa are two products, same family. Youth, Prison, and Marriage are likewise their own products. This catalog uses product_family for the line.

Dub/sub is not a new product. Language lives on the episode video, not as another product row. There is no catalog entry for "English AFS with French subs/dubs." This POC stores provider_asset_id + language_code and a download_url per language (no live Brightcove call). Copy (title, summary, description, episode titles) stays in the product's content_language. Requested language selects assets, not a translated product post.

Country is not a new product. A product is listed in markets (product_market). Caller country codes (ke, co) map to market.code through a country-market ACL created as we code. Optional filters (course type, audience, country, language) are not stored variant rows. If language is omitted, asset selection falls back to the product's content_language.

1.2 PROBLEM

The current content distribution solution is manifest-based, with hard-coded JSON files as the source of product information. The existing MyAlpha service APIs could not be effectively leveraged to serve this purpose:

- MyAlpha's service APIs are very tightly coupled to the WordPress ecosystem and data design language, making them convoluted, hard to navigate, and hard to customise for a limited purpose.
- MyAlpha's service APIs are slow: queries done in sequential PHP require multiple joins to source the relevant metadata for any given product.
- The manifest system is limited to whichever JSON files have been added to the repository, so there is no easy way to add new courses.
- The manifest system requires a lot of manual work to first source the course information, then copy it into the relevant key values.
- The manifest system only supports hard-coded URLs for video files. Since it is client-side there is no secure way of hooking it up to Vimeo and Brightcove to support additional quality profiles.

A solution is therefore needed that can:

- Store a list of products and product-related metadata, including support for external video hosts — not just URLs (first-party player streaming, HLS/DASH, download links, multiple quality formats).
- Return episode video in a requested language (asset language / Brightcove flag, not a second product).

- List products by marketed country or region (market ACL; not a new product per country).
- Provide an authenticated admin portal where an Alpha staff member can edit product details and episode language assets easily.

1.3 USERS

The primary users of the API are the same demographic as the WhatsApp Course Registration service. The primary users of the admin portal are Alpha or NAO staff members tasked with updating product information and episode video language assets.

User group	Surface	Role
Course hosts / helpers (API)	API (read)	Fetch up-to-date product data and episode video for sessions
Course guests (potentially, API)	API (read)	Access course materials as a guest
Alpha / NAO staff member	Admin portal	Create, edit, and maintain product catalog data

1.4 USER STORIES

API User

ID	Story	Trace
US-API-01	As a course host, I want a client to fetch the product data we have (optionally filtered by course type, audience, country, and language), so that I see up-to-date materials without a repository deploy.	
US-API-02	As a course host, I want that payload to include episode list, titles, and downloadable video in the requested language (or the product's content language if none specified), so that I can run the session with the right videos.	
US-API-03	As a helper, I want Getting Started and training videos (when the product has them), so that I can prepare before guests arrive. (Stretch)	Stretch
US-API-04	As a host or helper, I want results filtered by course type, audience, market, and video language, so that I only receive the product I registered for, with the right dub/sub.	

Portal User — Alpha Staff Member

ID	Story	Trace
US-POR-01	As an Alpha staff member, I want to authenticate through Alpha's existing Identity Provider via this product's portal BFF, so that I do not keep a separate password store.	
US-POR-02	As an Alpha staff member, I want to list products, so that I can see what the API will serve.	
US-POR-03	As an Alpha staff member, I want to filter the list by family and country/market, so that I can find the edition I am updating.	
US-POR-04	As an Alpha staff member, I want to create a new product (family, audience, markets, presentation fields), so that a new content edition can ship without editing JSON in a repo.	
US-POR-05	As an Alpha staff member, I want to edit an existing product (titles, intro, groups, episodes, Getting Started copy), so that corrections go live without a code change.	
US-POR-06	As an Alpha staff member, I want to set Brightcove id, language, and a download URL on an episode asset, so that a dub is another language row on the same video, not a new product.	
US-POR-07	As an Alpha staff member, I want saves to persist and be returned by the API, so that hosts receive the edit on the next fetch.	
US-POR-08	As an administrator, I want staff-only write access in the portal, so that only allow-listed accounts can change catalog data.	

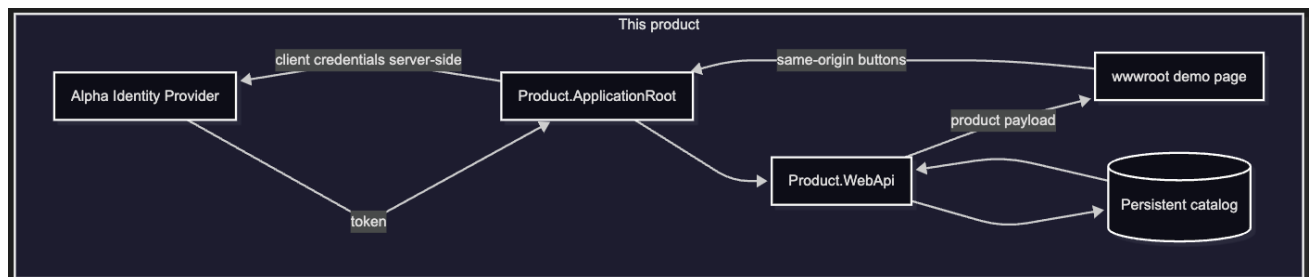
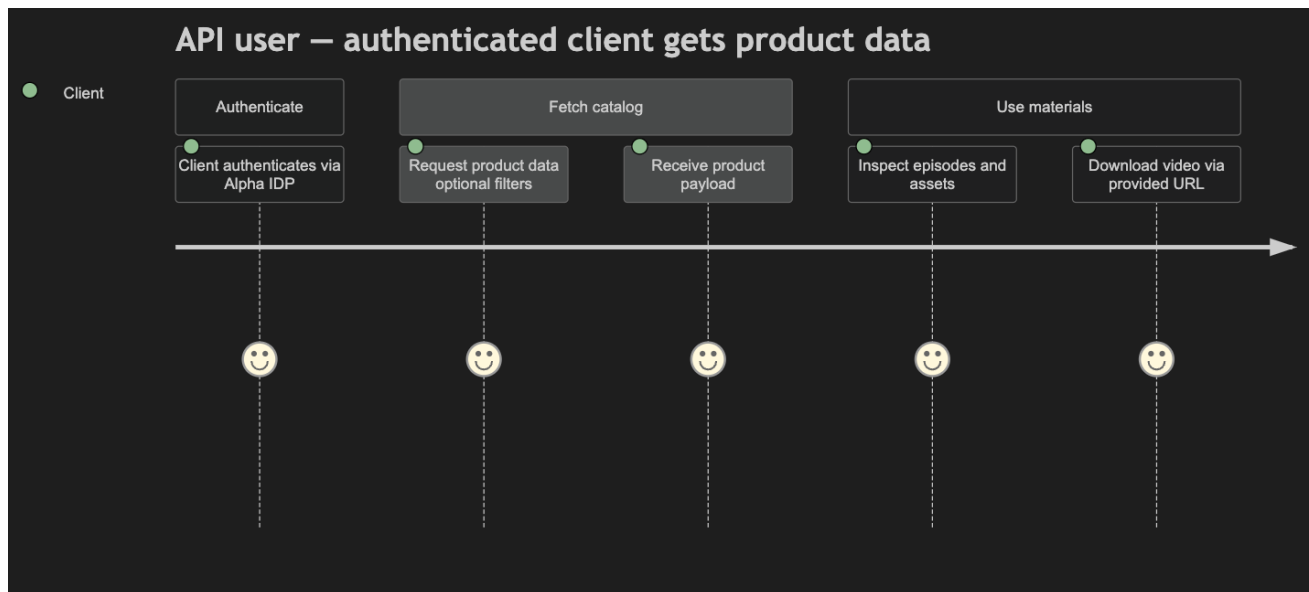
Portal stories above are sufficient for this stage. The portal is React Admin; staff authenticate via the portal BFF (OIDC, `alpha.idp.readwrite`), then the BFF issues the cookie session. Portal write is gated by an allow-list stub of IDP accounts. `Product.WebApi` uses client credentials (`alpha.idp.read`). "Evaluator vs administrator" labels are not a design constraint until they become a problem.

1.5 JOURNEYS

This exercise's screens are the admin portal and a `wwwroot` demo page on `ApplicationRoot`. WhatsApp Content Distribution is not in the runtime path and is not used to score the API.

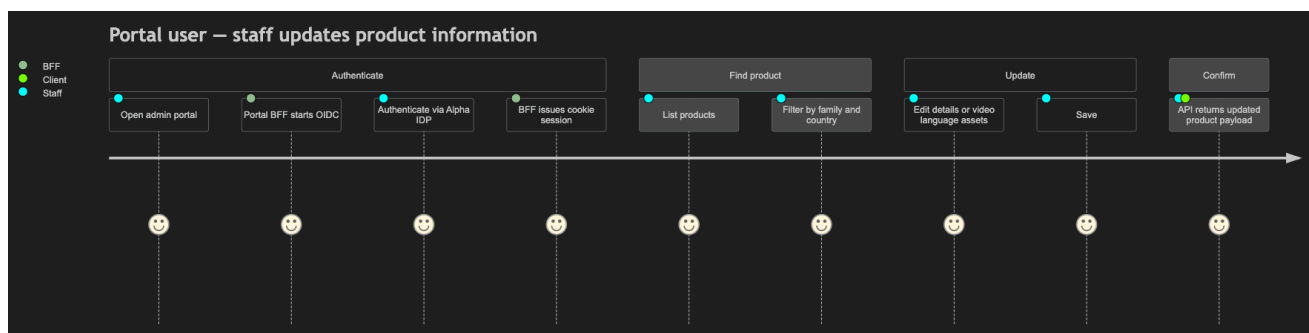
1.5.1 API User Journey

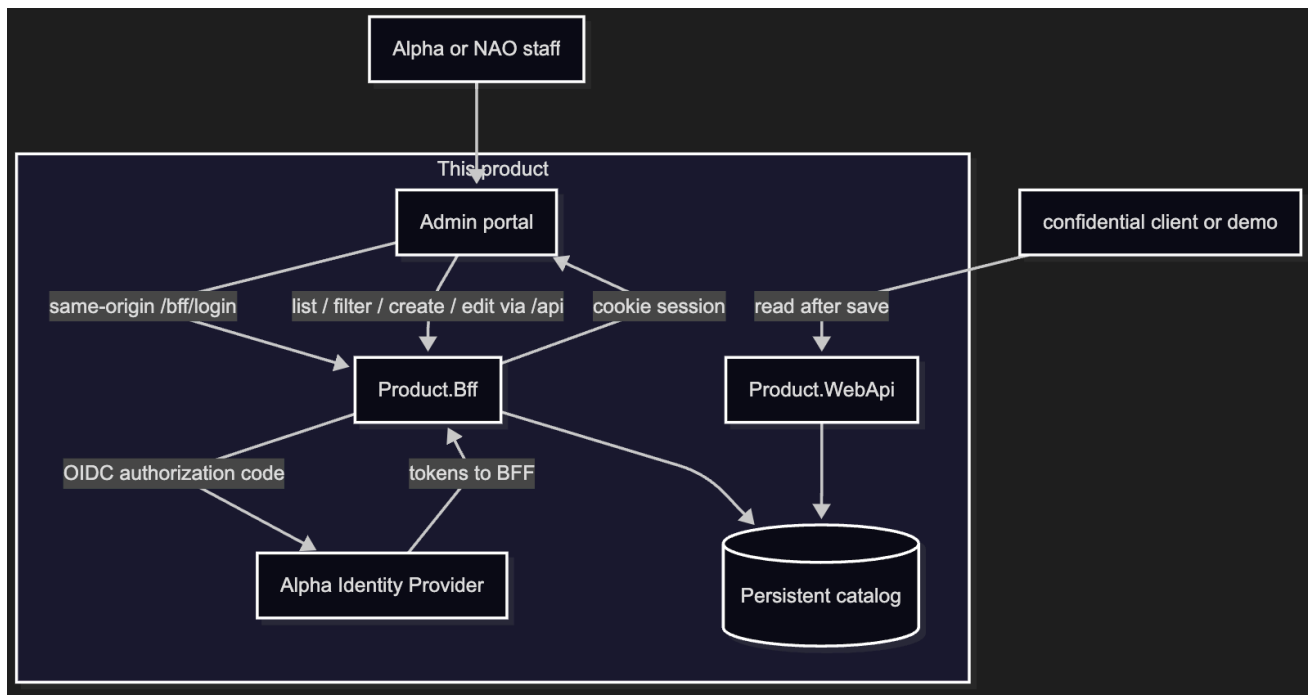
Primary path: the client authenticates (client credentials against IDP, scope `alpha.idp.read`), then fetches product data (optional filters). This POC's hands-on demo is the `ApplicationRoot` `wwwroot` page — buttons that call this host. A later consumer is the same read, if adapted.



1.5.2 Portal User Journey

Primary path: staff authenticate through the portal BFF (OIDC to IDP), get a cookie session, then find and edit a product. The next authenticated WebApi fetch returns the new version.





1.6 REQUIREMENTS

Functional — Product Catalog API

ID	Requirement	Trace
FR-API-01	An authenticated client can fetch the product data we store (baseline contract in 1.7). Optional filters: course type/family, audience, country/region, language. Those filters are not stored rows. Language selects episode assets (Brightcove flag); omitted language falls back to product content_language. Country maps through a market ACL implemented in code.	US-API-01, US-API-04
FR-API-02	The payload includes the catalog fields we persist: presentation, items (episodes/training), assets (including downloadable video for the requested or fallback language), tags, and markets.	US-API-02, US-POR-06
FR-API-03	Stretch. When a product has Getting Started content, the payload includes that copy and any training videos. Only if time remains.	US-API-03
FR-API-04	Results can be filtered by course type/family, audience, market (from country), and video language. Audience remains many-to-many; the API filters. Do not require exactly one audience per product.	US-API-04
FR-API-05	Product data is not served anonymously. Product.WebApi uses client credentials against IDP with scope alpha.idp.read (bearer JWT). No end-user login on the read API.	Scope
FR-API-06	After a portal save, the next authenticated fetch returns the updated product payload (no repo deploy).	US-POR-07
FR-API-07	The read contract starts as all product data in the system. It can be tightened later if a consuming service is adopted. It is not the WhatsApp Content Distribution manifest JSON.	Scope

Functional — Admin Portal

ID	Requirement	Trace
FR-POR-01	Alpha/NAO staff authenticate through Alpha's existing Identity Provider via Product.Bff (OIDC, then cookie session). No separate password store.	US-POR-01
FR-POR-02	Staff can list products.	US-POR-02
FR-POR-03	Staff can filter that list by family and country/market. Language is edited on episode assets, not as a product row.	US-POR-03
FR-POR-04	Staff can create a product (family, audience, markets, presentation fields) without editing JSON in a repo. A dub language is not a create-product action.	US-POR-04
FR-POR-05	Staff can edit an existing product: titles, intro, groups, episodes. Getting Started copy is stretch (if time).	US-POR-05

ID	Requirement	Trace
FR-POR-06	Staff can set Brightcove (or url) provider, provider_asset_id, language, and a download URL on an episode asset. Same Brightcove id may exist on more than one language row. One download URL per language this iteration.	US-POR-06
FR-POR-07	Saves persist and are what the API serves.	US-POR-07
FR-POR-08	Portal uses IDP interactive login with scope alpha.idp.readwrite through Product.Bff. Authorization is an allow-list stub of approved IDP accounts (not IDP roles). Product.WebApi uses alpha.idp.read.	US-POR-08
FR-POR-09	The portal is mobile-responsive.	Brief, scope

Functional — Exercise / Delivery

ID	Requirement	Trace
FR-X-01	The product is fully deployed and reachable by assessors (not local-only).	Brief
FR-X-02	One end-to-end workflow: staff edits a product in the portal; an authenticated client reads the updated product payload from the API.	Scope, brief
FR-X-03	Seed a small catalog: two content editions in one family (e.g. AFS and AFS Africa); two caller countries via market ACL; two languages on assets of one product (same Brightcove ids). Do not import production catalog or production media URLs/credentials.	Scope

Non-Functional

ID	Requirement
NFR-01	Persistent storage for products, items, and language-specific assets. Language variants are asset rows, not a stored JSON blob per query.
NFR-02	Consume Alpha Identity Provider as-is (dev discovery: https://dev.auth.alpha.org/.well-known/openid-configuration). Existing scopes alpha.idp.read and alpha.idp.readwrite. No IDP code, policy, roles, or user-store work. Demo OAuth clients may be registered; details in env after scaffold.
NFR-03	Dev/non-production auth only for this build.
NFR-04	Start from a baseline contract of all catalog data we store. Do not copy the WhatsApp Content Distribution manifest JSON. Tighten the contract later only if a consumer is adopted.
NFR-05	Secrets and production credentials stay out of source control and archives.
NFR-06	This service stores product catalog data only. No user profiles or PII. Identity data stays with IDP.
NFR-07	Each C# module has unit tests; inbound adapters and ApplicationRoot have acceptance tests. Portal has unit tests (Vitest).
NFR-08	Observability: Application Insights when ApplicationInsights__ConnectionString is set; otherwise stdout. Custom events for alertable outcomes. No tokens or IDP profile payloads in logs.

Stretch Goals

Getting Started content (payload, portal edit, training videos) is not required to call the exercise done. Include only if the core catalog API and portal ship first.

Explicitly Not Requirements for This Iteration

- Rebuilding WhatsApp Content Distribution, the WhatsApp worker, MyAlpha, Guest App, or Domain Services.
- Measuring this POC against the WhatsApp Content Distribution site or matching its manifest JSON.
- A new product row per dub/sub language; RTML extra posts for translated copy; MyAlpha country-site field-group overrides.
- Live Vimeo/Brightcove/HLS/DASH/first-party player integrations (fields may exist; no live host work). Caption/VTT management.
- OpenAPI/Swagger: UI at /swagger on ApplicationRoot. Paste a client-credentials bearer (scope alpha.idp.read). products.http remains the integrated test.

1.7 SCOPE

Baseline freeze for this exercise. The product is a small, authenticated product catalog API plus an admin portal, plus a wwwroot demo page on ApplicationRoot. WhatsApp Content Distribution motivated the problem (static JSON, no portal). This POC is not a drop-in for that manifest and is not scored against that site.

In Scope

- Persistent storage of products (content editions), items, and language-specific assets. Optional query filters (family/code, audience, country, language) are not a stored variant row.
- Authenticated read API returning a baseline contract: all product data in the system. Adjustable later; not the WhatsApp Content Distribution JSON shape.
- Filters: course type / family, audience (many-to-many; API filters; do not force one audience), country / region (market ACL written as we code), video language (asset language_code; if omitted, fall back to product content_language).
- Authenticated admin portal: React Admin (TypeScript, React, Vite). Full CRUD on catalog models. The SPA does not hold client secrets; staff authenticate through this product's portal BFF (OIDC to IDP, cookie session).
- Live deploy and one end-to-end workflow: staff edits a product, then the API returns the updated payload.
- Video asset record with more than a bare URL: Brightcove (or url) provider, provider_asset_id, language_code, and a download_url. Seed one download URL per language per episode (360p-style placeholder). Do not call live video platforms.
- Seed a small representative catalog: two products in one family (global AFS and AFS Africa); two caller countries mapping through an ACL; two languages on assets of one product (same Brightcove ids).
- Authenticate against Alpha's existing Identity Provider as standard OAuth/OIDC. No IDP service/policy/role work. Demo clients may be registered; ids/secrets go in env files after scaffold.
- Stack: one C# process (Product.ApplicationRoot) that plugs BFF and WebApi as adapter services, plus Product.ProductStore (Postgres repository). Portal: React Admin + TypeScript + Vite. Two repositories in the target operating model; this exercise colocates both under Solution/.

Authentication Intent (No IDP Service Work)

Use Alpha's self-hosted Duende IdentityServer. Authentication policies already live there. This build only consumes the provider as a standard OAuth/OIDC client. No changes to the IDP service (policies, users, themes, roles, claims, extra scopes, server configuration). Dev discovery: <https://dev.auth.alpha.org/.well-known/openid-configuration>

- IDP has role-based-access scaffolding only: no roles assigned, no role claims on tokens. Do not add roles in IDP. Scopes (existing): alpha.idp.read on Product.WebApi (client credentials, bearer); alpha.idp.readwrite on Product.Bff (interactive OIDC).
- Portal authorisation stub: configured allow-list of approved IDP accounts (subject or email). Not stored in the product database. Replace with IDP roles later without changing the catalog schema.
- Product.Bff: confidential OIDC client, cookie session (Duende BFF pattern). No client secret in the SPA. Staff-only: an IDP user not on the allow-list does not get portal access.
- Product.WebApi: bearer JWT from client credentials, scope alpha.idp.read. No end-user identity on read. The wwwroot demo page must not embed that secret in JavaScript; buttons call this host (same-origin / server-side token).

Baseline API Contract (All Product Data We Have)

The read API returns the catalog we persist, not a WhatsApp Content Distribution variant map. A payload includes: family and product (code, titles, summary, description, content_language, presentation fields); tags (including audience; many-to-many); markets the product is listed in; items (episodes and training: code, sequence, title, optional summary/grouping); assets (provider, provider_asset_id, language_code, download_url, capabilities, duration/size if stored). Query parameters filter that graph; they do not invent a second stored shape. Language omitted → use product content_language. Country → market via ACL coded during implementation.

1.8 KEY EXPERIENCE DECISIONS

Baseline architecture decisions. Domain model locked. ApplicationRoot / scopes / tests / privacy are in §3.

ID	Decision	Reasoning
D-ARCH-01	The product API is C#, same family as Alpha's Identity Provider.	If this becomes a product, the API team already knows the approach. Fits the current stack.
D-ARCH-02	The admin portal is React Admin on TypeScript + React + Vite. Full CRUD on catalog models.	Faster staff UI than a bespoke CRUD app. SPA has no client secret; it talks to the portal BFF.
D-ARCH-03	API and portal are two repositories (target operating model). This exercise colocates them under Solution/ as product (Product.sln) and product-portal.	API team owns C#; UI team owns the SPA.
D-ARCH-04	Persistence is PostgreSQL, accessed through Product.ProductStore (a repository).	Store is a driven adapter in role, not in project name.
D-ARCH-05	C# follows ports and adapters with Product.ApplicationRoot as the only process entrypoint. BFF and WebApi are adapter services. Domain owns entities and IProductRepository. No Product.Application project.	Catalog rules live in Domain; adapters stay thin.
D-ARCH-06	This POC does no work outside this project. WhatsApp Content Distribution is not a runtime client. Portal BFF is in this project. Read-API demo is ApplicationRoot wwwroot.	Not scoring a drop-in. A later consumer would need its own confidential BFF if it is a static site.
D-ARCH-07	Demo OAuth clients are registered for this POC; ids/secrets in env after scaffold. Existing scopes alpha.idp.read / alpha.idp.readwrite. Implementation is AI-assisted to stay inside 12 hours.	No IDP service/role/policy work.
D-ARCH-08	One process. ApplicationRoot maps Product.Bff and Product.WebApi on the same host. Both call Domain / ProductStore in-process.	No second host, no HTTP hop between adapters.
D-ARCH-09	WebApi = client credentials + alpha.idp.read. Portal = interactive OIDC + alpha.idp.readwrite, then an allow-list stub.	Scopes already exist. Roles do not. Stub is replaceable when IDP roles ship.
D-DOM-01	A product is one content edition. Dub/sub is not a new product: Brightcove language flag on the episode asset. Copy stays on the product.	Locked 2026-09-01. Avoids rebuilding MyAlpha's separate graphs for country override and RTML copy.
D-DOM-02	Country is market listing + an ACL created in code. Optional filters are not stored variant rows. Language omitted → product content_language. Audience stays many-to-many.	Avoids storing one variant row per language/country combination. ACL is not a dossier table.

2. SYSTEM DESIGN

2.1 EXPLANATIONS & ARCHITECTURE NARRATIVE

Three existing or neighbouring systems sit around this product:

- Alpha Identity Provider (IDP) — self-hosted Duende IdentityServer, C#. Consumed here; not changed. Demo clients may be registered; details in env after scaffold. No role claims today.
- WhatsApp Content Distribution — JavaScript / Astro static client. Motivation only for this POC. Not rebuilt, not integrated, not used to score the API.
- MyAlpha BFF — cookie/OIDC BFF pattern and wwwroot demo page reference. Not rebuilt or modified.
- Core Services (Profile) — ApplicationRoot convention: entrypoint plugs adapters as services. Not rebuilt or modified. The double-nesting and reverse-proxy pattern is explicitly not copied.
- This product — one C# process (Product.ApplicationRoot) plus a React Admin portal (TypeScript, Vite) plus a wwwroot demo page.

The application core is the system of record for product data only (no user records). Product.Bff authenticates staff (OIDC cookie, alpha.idp.readwrite, allow-list stub) and exposes CRUD for React Admin. Product.WebApi exposes read-only product payloads (client credentials, alpha.idp.read). Both are adapter services on ApplicationRoot, same process. The portal SPA and the wwwroot demo never hold a client secret.

Ports and adapters structure:

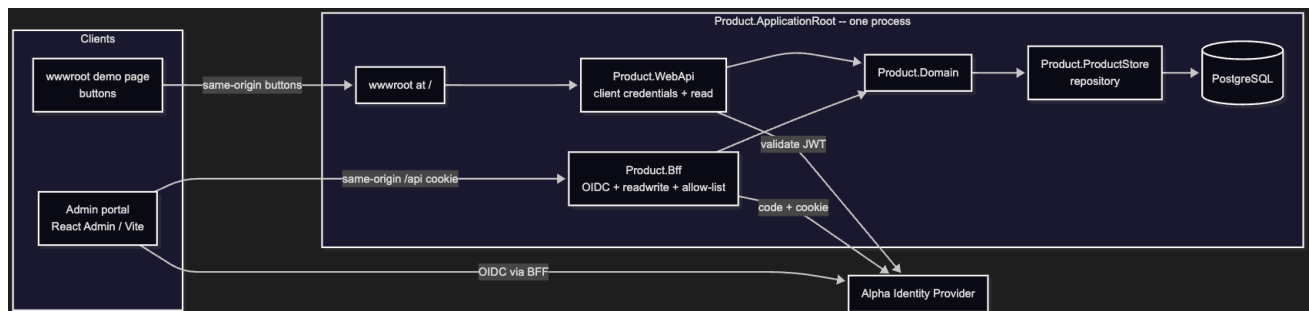
- Product.ApplicationRoot is the only process. Program calls AddProductModules: ProductStore first (so Bff and WebApi can inject IProductRepository), then Bff and WebApi. Adapters are plugged as services on the same host.
- Product.Domain owns entities and the IProductRepository port. There is no Product.Application (use-case) project.
- Product.ProductStore is the Postgres repository. ProductStoreAdapter registers scoped ProductDbContext, scoped DbContextOptions, and scoped IProductRepository (not singleton).
- Product.Bff and Product.WebApi are inbound adapters. They map HTTP on the same host (path prefixes). They do not own catalog rules and do not HTTP-call each other.

Taken from Core Services Profile: ApplicationRoot as composition root; WebApi as an adapter service. Not taken: double-nested src/, inner Kestrel per adapter, reverse proxy, Driven.*/Driver.* names, Service Bus, cron.

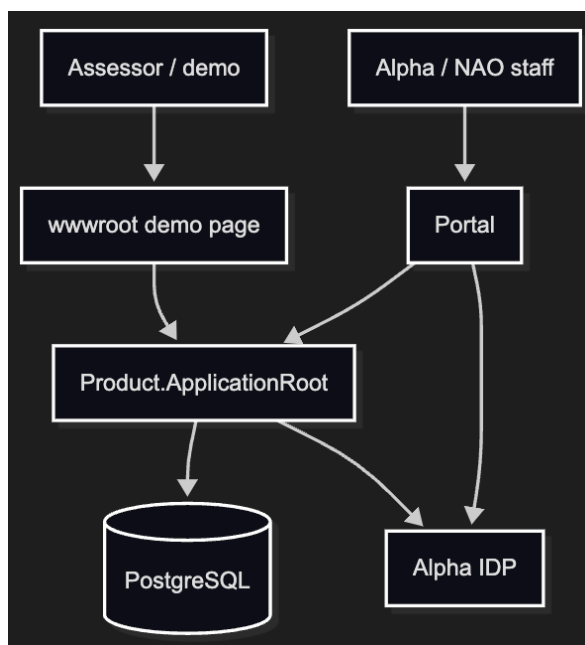
Taken from MyAlpha BFF: cookie + OIDC, /bff/login, __Host-cookie, AsBffApiEndpoint. Not taken: WordPress/MySQL, HMAC, token-exchange, YARP to remote Alpha APIs.

Auth: WebApi = client-credentials bearer + alpha.idp.read. Portal = interactive OIDC + alpha.idp.readwrite + allow-list stub. Demo clients in env after scaffold.

2.2 ARCHITECTURE DIAGRAM



2.3 CONTEXT DIAGRAM



2.4 COMPONENT DIAGRAM & SOLUTION STRUCTURE

One C# process. Names as below (no Driven.* prefixes). Two repositories in the target operating model; colocated under Solution/ for this exercise.

Solution/	
product/	C# (API team) — Product.sln
src/	
Product.ApplicationRoot/	entrypoint: one process, plugs adapters; wwwroot demo at /
Product.Domain/	entities + IProductRepository port
Product.Bff/	inbound adapter: Duende BFF, OIDC, CRUD
Product.WebApi/	inbound adapter: client credentials, read catalog
Product.ProductStore/	Postgres repository (implements IProductRepository)
tests/	
Product.Domain.Tests/	unit
Product.ProductStore.Tests/	unit (+ store acceptance against Postgres)
Product.Bff.Tests/	unit
Product.Bff.Acceptance.Tests/	portal BFF HTTP
Product.WebApi.Tests/	unit
Product.WebApi.Acceptance.Tests/	client-credentials read HTTP
Product.ApplicationRoot.Tests/	composition unit
Product.ApplicationRoot.Acceptance.Tests/	staff-edit then API-read (FR-X-02)
product-portal/	Vite + React Admin (Vitest unit)

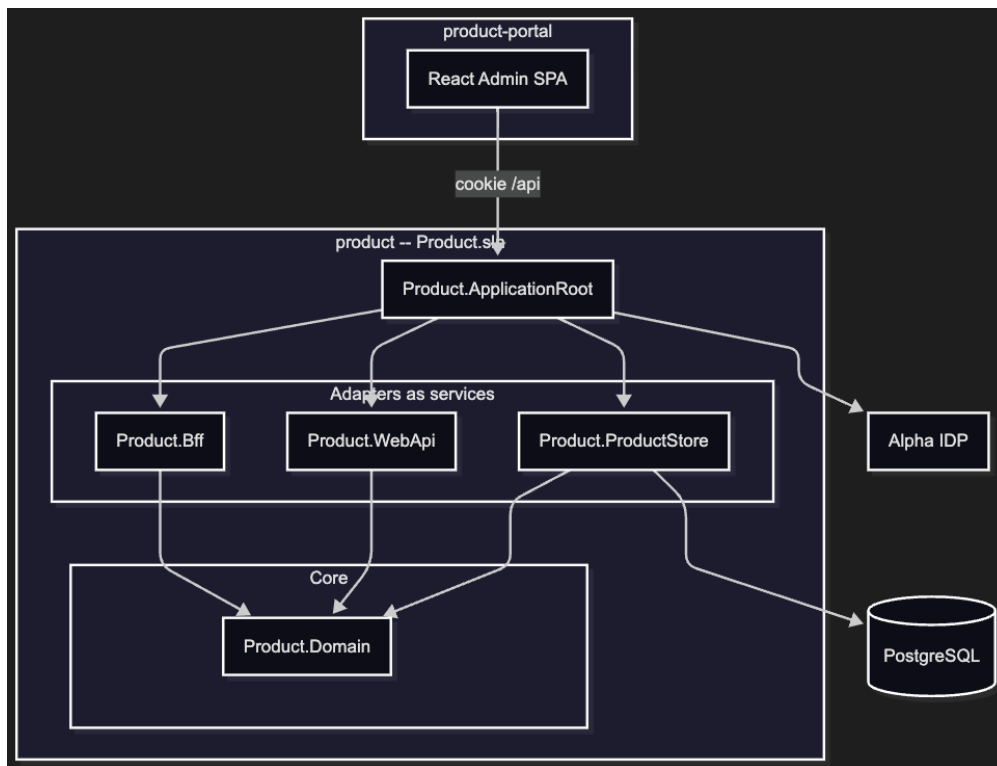
Product.ApplicationRoot: Program/host builder calls AddProductModules (ProductStore, then Bff, then WebApi). Registers Application Insights when the connection string is set; always keeps stdout logging. One Kestrel listener. BFF and WebApi contribute route groups; they do not start their own web hosts. wwwroot at the root of the host (/): a single static demo page with buttons to exercise the read API. No client secret in that page; same-origin / server-side token.

Product.Bff: Cookie + OIDC to IDP, scope alpha.idp.readwrite, UseBff, MapBffManagementEndpoints. Named cookie/OIDC schemes (ProductBffCookie / ProductBffOidc) so they cannot satisfy /product bearer routes. After authentication, allow-list stub (configured IDP account ids / emails). Empty list denies everyone. CRUD under /api (React Admin / ra-data-simple-rest). AsBffApiEndpoint (CSRF X-CSRF: 1). Omitted ClientId skips OIDC so tests can start.

Product.WebApi: Inbound adapter; ProductWebApiAdapter maps /product on ApplicationRoot (same host; no inner Kestrel). JWT bearer from client credentials, scope alpha.idp.read. Swagger UI at /swagger: two documents — Product API (bearer) and Portal BFF (/api, cookie + X-CSRF: 1). products.http is the integrated test for the read API. Read routes return the baseline product contract. No create/update/delete.

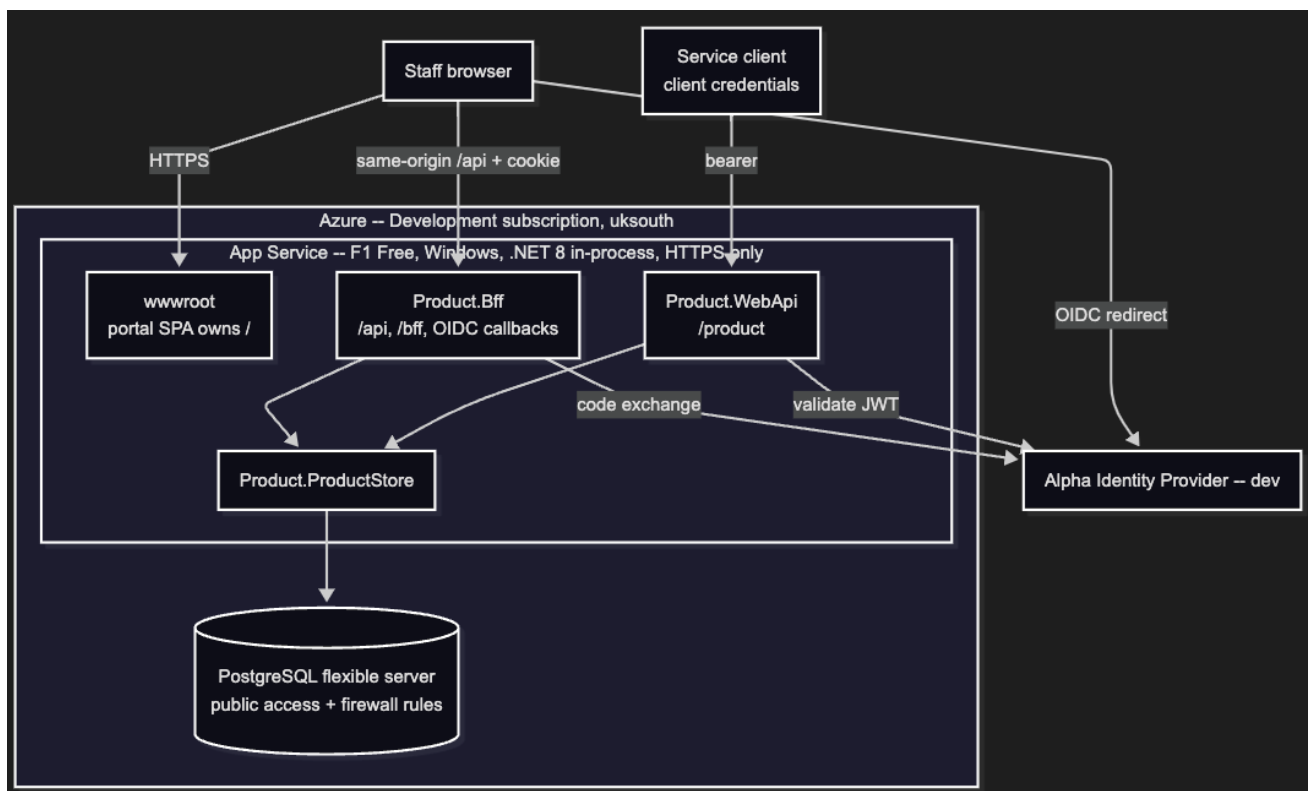
Product.ProductStore: ProductStoreAdapter registers scoped ProductDbContext, scoped DbContextOptions, and scoped IProductRepository. AddProductStore is the composition-root façade; ApplicationRoot calls it via AddProductModules before Bff and WebApi.

product-portal: Scaffolded with Vite react-ts + React Admin + Vitest, pnpm only. Resources for BFF models: lists show related names (not FK ids); create/edit pick related rows in a native <dialog>. Dev SPA is the Vite origin root (/). Sign-in is a full-page /bff/login: Product.Bff OIDC to the IDP, callback /signin-oidc, cookie session. After login, a spinner stays up while checkAuth probes the allow-list (GET /api/languages). Allow-listed accounts open products. A 403 shows an Unauthorized page with Log out. Logout navigates to /bff/logout?sid=... from bff:logout_url. Custom same-origin dataProvider (cookie + X-CSRF: 1).



2.5 DEPLOYMENT DIAGRAM

One C# deployable (Product.ApplicationRoot) that also serves the portal SPA from its own wwwroot. This is a constraint, not a convenience: the portal is a same-origin client of Product.Bff (cookie session plus X-CSRF), and the OIDC callbacks /signin-oidc and /signout-callback-oidc land on the portal's own origin. The SPA and the BFF therefore have to answer on one host. Shipping the built SPA inside the API's wwwroot achieves that with no reverse proxy, no CORS, and no third-party cookies.



Path Ownership

Path	Served by
/ plus /assets/*, /favicon.svg	Portal SPA, from wwwroot
/api/*, /bff/*	Product.Bff (cookie + X-CSRF)
/signin-oidc, /signout-callback-oidc	Product.Bff (OIDC callbacks)
/product/*	Product.WebApi (bearer token)

Portal routes are hash-based (/#/products), so every portal URL reaches the host as / and no server-side SPA fallback route is required. The ApplicationRoot demo page (index.html, app.js, styles.css) stays in source but is excluded from the deployed payload — the SPA owns /, and two index.html files cannot both hold it.

Deployment Steps

Ordering matters: the SPA must be built before the publish payload is assembled.

1. Build the portal SPA. In Solution/product-portal, install with the committed lockfile and run the production build to produce dist/. No origin is baked in — the bundle calls /api on whatever host serves it.
2. Publish Product.ApplicationRoot in Release configuration to a staging directory. web.config is generated automatically with hostingModel="inprocess" and processPath="dotnet".
3. Merge the SPA into the payload. Delete the demo page files from the published wwwroot, then copy the contents of dist/ in, so the SPA owns /.
4. Apply the database schema and seed to the PostgreSQL flexible server.
5. Set App Settings on the App Service: ConnectionStrings__ProductStore, Bff__ClientId, Bff__ClientSecret, Bff__CountryCode, and at least one Portal__AllowList__0 entry — without it the allow-list stub grants nobody portal access.
6. Register the redirect URIs on the portal demo client in the IDP: <host>/signin-oidc and <host>/signout-callback-oidc. The App Service default hostname carries a random suffix, and the registered URI must match exactly.
7. Deploy the merged publish output to the App Service as a zip whose contents sit at the archive root.
8. Verify: / returns the portal shell; /product/languages returns 401; /bff/user returns 401; then complete an interactive sign-in through the IDP in a browser.

Steps 1 to 3 and 7 are currently manual — the C# solution is published from the Rider IDE, and the SPA merge is a separate manual build-and-copy. This is the weakest part of the setup: the merge is easy to forget, which would silently deploy an API with a stale or missing portal. Intended target: a pipeline owns the whole sequence — build both artefacts, run tests, assemble the merged payload, deploy to a slot, smoke-test, and swap.

2.6 DATABASE SCHEMAS

PostgreSQL database product_service (local). Apply Solution/Data/product_schema.sql. Validate with Solution/Data/product_schema_smoketest.sql. Local example graph: Solution/Data/seed_product_data_example.sql. Country–market ACL is code, not a table.

Schema DDL is in Solution/Data/product_schema.sql. The schema opens with DROP TABLE ... CASCADE — correct for a POC that reseeds freely; unusable in production without a migration tool. Versioned migrations are the first item of production work (Phase 0). See §3.6 and §4.3.1.

2.7 DATA MODEL

The data model's primary purpose is a product catalog: store editions, items, and assets, and return that graph on the read API. WhatsApp Content Distribution motivated the problem; the manifest JSON is not projected. Intent is taken from MyAlpha (separate course posts for different films; language on the video; country as a listing) without rebuilding WordPress graphs (country field-group overrides, RTML copy posts, (id, blog_id, language) tuples).

The diagram illustrates the database schema for OpenStreetMap, showing the relationships between various tables and their attributes.

Tables and Attributes:

- language**: PK text code, boolean is_active
- product_family**: PK uuid id, UK text code, text name, text summary, integer sequence
- product**: PK uuid id, FK uuid family_id, UK text code, text title, text summary, text description, FK text content_language
- product_item**: PK uuid id, FK uuid product_id, text kind, text code, integer sequence, text title, text summary, text grouping, boolean is_optional
- product_tag**: PK uuid product_id, PK uuid tag_id
- tag**: PK uuid id, text category, text code, text name, boolean is_public, integer sequence
- market**: PK text code, text kind, text name
- product_market**: PK uuid product_id, PK text market_code, date launched_on
- asset**: PK uuid id, FK uuid product_id, FK uuid item_id, text role, text kind, FK text language_code, text title, text group_code, text provider, text provider_asset_id, text stream_url, text download_url, boolean allow_stream, boolean allow_download, integer duration_seconds, bigint file_size_bytes
- asset_market**: PK uuid asset_id, PK text market_code

Relationships:

- language** to **product_family**: copy language
- product_family** to **product**: editions
- product** to **product_item**: items
- product** to **product_market**: listed in
- product** to **asset**: all assets
- product_item** to **asset**: item assets
- product_tag** to **product**: tagged
- product_tag** to **product_item**: tags
- market** to **product_market**: hosts
- market** to **asset_market**: targets
- asset** to **asset_market**: marketed in

Table	Definition	Why included
language	Closed list of BCP-47 content-language codes Alpha uses.	FK target for product copy and assets. Prevents en/EN/en-gb drift. Not a locale directory.
market	A place a product can be listed: a country or a region, one code (za, ssa).	Staff have one country/region slot. Caller country codes (e.g. ke, co) are not stored as markets 1:1; a country–market ACL implemented in code maps caller country → market.code.
product_family	The course line shared by content editions (Alpha Film Series, Youth, Marriage Course).	Groups sibling products. Africa editions typically use a distinct product.code.
product	One content edition: copy in one language, its own items and assets. Different films = new row.	Catalogue unit. AFS and AFS Africa are siblings. A French dub is not a third product.
tag	One vocabulary of suitability answers (audience, context, format).	Flattened MyAlpha coursequestion answers. Audience lives here. category is normalised (lowercase); portal first ships a small hard-coded set.
product_tag	Link: this product has this tag.	Many-to-many, including audience. The API can filter by audience. No unique-audience constraint.
product_market	Link: this product is listed in this market. Absence = not listed there.	Not a country copy override. No launched/pilot/withheld status this round.
product_item	An ordered session in a product: an episode or a training session.	Same shape, one table. Items are not shared: two courses using the same video still get two product_item rows.
asset	One watchable, readable, or downloadable file (video, PDF, image, ...), with hosting metadata.	Dub/sub of an episode is another asset row (same provider_asset_id, different language_code), not another product.
asset_market	Optional restriction: this asset is marketed only in these markets.	No rows = everywhere. Exact match intended.

Product & Technical Dossier · Confidential

Table	Column	Definition	Why included
product_family	code	Stable slug of the line (alpha-film-series). Unique.	Family identity. Africa editions use product.code.
product_family	sequence	Sort order among families.	Replaces MyAlpha menu_order at family level.
product	code	Stable slug (alpha-film-series-africa). Unique.	Human key when the edition differs from the family slug.
product	content_language	FK to language. Language of title/summary/description/episode titles.	One product = one language of text. Video language is on asset. No RTML copy post.
tag	category	Bucket (audience, context, format). Normalised lowercase.	Distinguishes audience from other answers.
tag	is_public	If false, never shown in public listings.	Youth-safeguarding. Unused by demo; kept for a later public listing.
product_item	kind	episode or training.	One table for two session types.
product_item	grouping	Free-text label (weekend, module-2).	Hint for API/UI grouping. Not a group entity.
product_item	is_optional	If true, omitted from completeness counts.	Partial-dub reporting.
asset	item_id	FK to product_item, or NULL.	NULL = product-level (hero, promo, host guide). Set = hangs off an episode/training.
asset	role	main_video, supporting, material, thumbnail, promo_video, promo_banner, hero_image.	One table instead of material/promo entities.
asset	language_code	FK to language, or NULL.	Brightcove language flag. Requested language selects this; omitted → product content_language. NULL = language-neutral (logo).
asset	group_code	Shared key for language variants of one logical material file.	EN/FR discussion guide without a join table. Video dubs share provider_asset_id instead.
asset	provider	brightcove, vimeo, or url.	Brightcove is the pattern for this catalog. Vimeo remains a value.
asset	provider_asset_id	Id on Brightcove (or Vimeo).	Same id across dubbed languages. Production requests that id with a language flag.
asset	download_url	File URL for download.	One URL per language this iteration (POC does not call Brightcove).
asset	allow_stream	Whether it may be played in a player.	Capability. Not entitlement.
asset_market	market_code	FK to market. Part of PK.	Where this asset may be marketed. Omit all rows ⇒ everywhere. Exact match intended.

2.8 DATA FLOWS

Primary flows: staff (allow-listed) authenticate via Product.Bff OIDC (alpha.idp.readwrite); BFF issues cookie session; React Admin CRUD → /api → Domain → ProductStore. Confidential client (or ApplicationRoot wwwroot demo using a server-side token) calls Product.WebApi with client-credentials bearer (alpha.idp.read); Domain returns the baseline product contract. Country ACL (in code) and language fallback (product content_language) live in Domain. See journeys in §1.5.

2.9 APIS & CONTRACTS

Product.WebApi (prefix /product, read-only). Auth is client credentials, alpha.idp.read, named JWT bearer scheme. Handlers live in Product.WebApi/Endpoints. Swagger UI is /swagger (anonymous page; dropdown for Product API bearer and Portal BFF /api with cookie + X-CSRF: 1). Portal CRUD remains Product.Bff /api. GET reuses Product.WebApi handlers; writes are BFF-local. Cookie + alpha.idp.readwrite + allow-list; X-CSRF: 1. All live on Product.ApplicationRoot.

Method	Path	Notes
GET	/product/languages	Lookup list
GET	/product/languages/{code}	Lookup
GET	/product/markets	Lookup list
GET	/product/markets/{code}	Lookup
GET	/product/families	Lookup list
GET	/product/families/{id}	Lookup
GET	/product/tags	Lookup list
GET	/product/tags/{id}	Lookup
GET	/product/products	Baseline product payloads. Query: courseType, audience, country, language. Unknown country → 400. country maps via Domain country–market ACL. Omitted language → product content_language for assets.
GET	/product/products/{id}	Full payload; optional language
GET	/product/products/code/{code}	Full payload by product code; optional language
GET	/product/items	Optional productId
GET	/product/items/{id}	Item row
GET	/product/assets	Optional productId, itemId
GET	/product/assets/{id}	Asset row
GET	/product/product-tags	Optional productId
GET	/product/product-markets	Optional productId
GET	/product/asset-markets	Optional assetId

Product GET payloads include family, tags, markets, items (with language-selected assets), and product-level assets. Write methods are not mapped on WebApi.

2.10 INTEGRATIONS & DEPENDENCIES

System	Role	Work in this exercise
Alpha Identity Provider (C# / Duende)	AuthN	Consume the service. Existing scopes alpha.idp.read / alpha.idp.readwrite. No role claims. Demo clients registered; details in env after scaffold.
PostgreSQL	Persistence	Owned by the C# application.
WhatsApp Content Distribution (JS / Astro)	Motivation	Not rebuilt, not integrated, not used to score this POC.
ApplicationRoot wwwroot demo	Read demo	Single page with buttons at /. Same-origin / server-side token. No client secret in the page.
Admin portal (React Admin / Vite / TS)	Staff UI	Separate repo. Same-origin CRUD against Product.Bff. Authenticate via BFF OIDC. Served under a path such as /admin.
MyAlpha BFF	Reference	Cookie/OIDC BFF pattern and wwwroot demo family. Not a runtime dependency. Not modified.
Core Services (Profile)	Reference	ApplicationRoot composition. Not a runtime dependency. Not modified. Do not copy inner-host reverse proxy.

System	Role	Work in this exercise
Application Insights	Observability	Standard Alpha approach. Connection string from env. Fallback: stdout if unset. Logs + custom events for alerts.

No MyAlpha, WordPress, live Vimeo, live Brightcove, or Mixpanel integrations in this iteration. Brightcove is modelled as provider + provider_asset_id + language_code only.

2.11 AUTHENTICATION & ACCESS

See §1.7 (Authentication intent) and §3.3 (Security model). The IDP service is consume-only; demo clients go in env after scaffold. No user rows are stored in Postgres. The wwwroot demo must not hold the WebApi client secret in the browser.

Surface	Grant type	Scope	Token / credential
Product.WebApi	Client credentials	alpha.idp.read	Bearer JWT
Product.Bff / portal	Authorization code (OIDC) via Duende BFF	alpha.idp.readwrite	Server-side cookie; SPA has no client secret

2.12 INFRASTRUCTURE & ENVIRONMENTS

Piece	Baseline
Product.ApplicationRoot	One C# process; plugs Bff + WebApi + ProductStore; serves the portal SPA at / from wwwroot
Product.Bff	Adapter service: Duende BFF, OIDC cookie, CRUD /api
Product.WebApi	Adapter service: JWT bearer, read-only catalog routes
Product.ProductStore	Postgres repository
Portal	React Admin, TypeScript, Vite; source in product-portal; built to dist/ and shipped inside ApplicationRoot wwwroot
Database	PostgreSQL
Auth	IDP dev; scopes alpha.idp.read / alpha.idp.readwrite; portal allow-list stub; demo client ids in env
Observability	Application Insights when configured; otherwise stdout
Exercise layout	Colocate product and product-portal under Solution/

Hosting: Azure, in the Development subscription, region uksouth — one App Service (Windows, .NET 8, in-process, HTTPS-only) on an F1 Free plan, plus one PostgreSQL flexible server (v18, Burstable B2s) with public access restricted by firewall rules to the deployer's address and the App Service outbound addresses. No VNet integration and no custom domain — F1 supports neither. Run cost is the database only; the compute tier is free.

The single-host layout is what keeps this cheap. Hosting the portal as its own Azure resource would require reunifying the origin, since the portal cannot authenticate cross-origin without third-party cookies. Static Web Apps cannot do that alone: linked backends are Standard-plan only and restricted to the /api route prefix, so /bff and the OIDC callbacks could not be proxied. The workable separated option is Front Door in front of a Static Web App and the App Service — worth it when the portal needs an independent release cadence, WAF, or CDN caching, and not before.

Pipelines: not yet built; deployment is manual today (see §2.5). Target: a single pipeline that builds both artefacts, assembles the merged payload, and deploys to a slot with a smoke test before swap.

Environments: one. There is no staging environment, and the deployed app runs with ASPNETCORE_ENVIRONMENT=Development — see §3.6.

3. QUALITY AND SECURITY

3.1 TESTING

Each C# module has a paired test project. Unit tests sit on every module. Acceptance tests sit on the HTTP adapters and ApplicationRoot (the surfaces a caller can hit).

Project	Kind	What it proves
Product.Domain.Tests	Unit	Entities, resolve query (course type / audience / market ACL / asset language), no HTTP.
Product.ProductStore.Tests	Unit + store acceptance	Repository against Postgres (or a test database). Persistence of products/items/assets.
Product.Bff.Tests	Unit	Allow-list stub, CSRF/BFF wiring helpers, mapping to Domain.
Product.Bff.Acceptance.Tests	Acceptance	Interactive cookie session, alpha.idp.readwrite, allow-listed account can CRUD; non-listed IDP user is denied.
Product.WebApi.Tests	Unit	Return baseline product contract; reject missing scope.
Product.WebApi.Acceptance.Tests	Acceptance	Client-credentials bearer with alpha.idp.read can read; write methods are absent/forbidden; wrong/missing token fails.
Product.ApplicationRoot.Tests	Unit	Adapters are plugged; one host.
Product.ApplicationRoot.Acceptance.Tests	Acceptance	FR-X-02: allow-listed staff save via BFF; confidential client (or demo host) reads the updated payload from WebApi.
product-portal (Vitest)	Unit	React Admin resources/forms. Browser E2E only if time; otherwise covered by Bff/ApplicationRoot acceptance.

Exercise success does not require full coverage. Priority: Domain resolve + ProductStore persistence + WebApi read acceptance + one portal-save-then-API-read path.

3.2 OBSERVABILITY

Use Alpha's usual Application Insights integration (same family as Core Services / MyAlpha BFF). Scaffold in Product.ApplicationRoot. Configuration: connection string from environment, not source control. If missing or empty, log to stdout only (local/dev fallback). When set, log to Application Insights and stdout.

```
ApplicationInsights__ConnectionString=<instrumentation connection string>
```

Kind	Use
Request / dependency traces	Default AI ASP.NET telemetry
Logs (ILogger)	Request path, product id/code, outcome. Never access tokens, ID tokens, or IDP profile payloads.
Exceptions	Unhandled and handled failures we care about
Custom events	Conditions we may alert on — see below

Custom Events (Alert Candidates)

Event name	When
portal.allow_list.denied	Authenticated IDP user is not on the staff allow-list
auth.failed	Missing/invalid token or cookie on WebApi or BFF
catalog.resolve.missed	Resolve query matched no product/assets
product.saved	Staff persisted a catalog change (telemetry; alert only if volume is wrong)

Alert rules themselves live in Application Insights / Azure Monitor later; this POC emits the events and logs. No Mixpanel. No user-analytics store.

3.3 SECURITY MODEL

Authentication (IDP, Consume-Only)

Surface	Grant	Scope	Token
Product.WebApi	Client credentials	alpha.idp.read	Bearer JWT
Product.Bff / portal	Authorization code (OIDC) via Duende BFF	alpha.idp.readwrite	Server-side cookie; SPA has no client secret

Scopes already exist on IDP. This build does not add scopes or roles. Demo OAuth clients may be registered; ids/secrets live in env after scaffold, not in source control.

Authorization (This Product, Not IDP Roles)

IDP does not return role claims. After a successful portal login, an authentication/authorization stub checks the IDP account (subject or email) against a configured allow-list. Only listed staff get portal CRUD. The list is configuration (environment), not a table in Postgres.

WebApi authorization is scope: alpha.idp.read on a client-credentials token. No user allow-list on the read API. Brief evaluator vs administrator labels: treat as semantics to ignore until they become a problem.

Data in this service: product catalog. Tokens are validated and discarded; they are not stored as user records.

3.4 THREAT & RISK ANALYSIS

ID	Threat	Impact	Mitigation this iteration	Residual
T-01	wwwroot demo or SPA given a WebApi client secret	Secret extracted from the JS bundle	Demo page and portal do not hold client credentials. Buttons call this host (same-origin / server-side token). Confidential client stays on the server or in env for tests.	High if someone later wires the secret into static JS.
T-02	Any IDP user with readwrite reaches portal CRUD	Catalog defacement	Allow-list stub after OIDC. Unlisted accounts denied.	List is only as good as config hygiene; not real RBAC.
T-03	Allow-list committed to git	Staff emails/subjects in the repo	Config/env, not source. NFR-05.	Mis-config in a shared env file.
T-04	Bearer token used on BFF cookie routes (or the reverse)	Privilege mix-up	Separate route groups; WebApi requires client-credentials + read; BFF requires cookie + readwrite + allow-list.	Implementation must keep schemes from leaking across prefixes.
T-05	CSRF on cookie CRUD	Write as the staff user	Duende BFF anti-forgery (X-CSRF / AsBffApiEndpoint), __Host- cookie.	Depends on correct BFF setup.
T-06	IDP account takeover of an allow-listed staff user	Full catalog write	IDP owns passwords/MFA (out of this service). Staff-only list limits who can be a portal user.	Same as any staff SSO.
T-07	Catalog download URLs leaked	Video files fetched without our API	Seed uses placeholder URLs. API still requires alpha.idp.read. No public anonymous catalog.	Media hosting is outside this POC.
T-08	Allow-list stub mistaken for production RBAC	Over-trust	Documented in §3.6; replace with IDP roles later; no user table to migrate.	Process risk.
T-09	Logs store tokens or IDP profile claims	PII in logs	§3.2: do not log tokens or profile payloads.	Operator error.

3.5 DATA & PRIVACY DECISIONS

- Stored here: product catalog only (families, products, items, assets, markets, tags, languages). No user table, no host/guest PII, no token store.
- Identity: IDP remains the system of record for people. Access tokens and ID tokens are used to authenticate requests and are not persisted as user data in this service. They fall under existing IDP data-privacy policies.
- Portal: interactive login is staff-only (allow-list). That limits which IDP accounts can present a user token to this app.
- WebApi: client credentials — no end-user identity on read. Confidential client, not a browser.
- Allow-list: account identifiers in configuration. Treat as operational config, not catalog data.

3.6 KNOWN LIMITATIONS

Authentication / Authorisation

- Portal authZ is an allow-list stub, not IDP roles. Full RBAC waits on IDP (Phase 2).
- WhatsApp Content Distribution is not a consumer in this POC. A later static consumer would need its own confidential BFF.
- React Admin is dense on small phones (FR-POR-09 is best-effort).
- Getting Started remains stretch — not built.
- Alert rules in Azure Monitor are not designed in this POC; events and logs are emitted so they can be wired later.
- Country–market ACL is implemented in code when built; not specified as a schema table.

Hosting

- No server-side SPA fallback route needed (React Admin uses HashRouter; portal routes live after # and the host sees only /). A hand-typed non-hash path such as /products returns 404, but the portal never generates that URL.
- The deployed app runs with ASPNETCORE_ENVIRONMENT=Development — this activates UseForwardedHeaders (scoped to Development in Program.cs) and developer exception pages. Forwarded-header handling should move out of the Development branch.
- Bff__ClientSecret is a plaintext App Setting, not a Key Vault reference with a managed identity.
- F1 Free cannot enable Always On, so the app cold-starts after idle. Duende's key ring and any in-flight OIDC state do not survive a recycle, which can make an interactive sign-in fail once after a quiet period.
- One environment, no staging and no deployment slot. Deployments go straight to the live host.
- The portal ships as a single ~1.4 MB JS chunk (~409 kB gzipped). Acceptable for a POC; code-splitting is the first optimisation if the portal takes real traffic.

Data Model & Schema

- product_schema.sql is drop-and-recreate, not a migration. Opens with DROP TABLE ... CASCADE. Correct for a POC that reseeds freely; unusable in production. Versioned migrations are the first item of production work (Phase 0).
- product_schema_smoketest.sql is not idempotent against a seeded database. Its cleanup deletes markets (za, ssa); market is ON DELETE RESTRICT and a seeded product_market references both, so the smoke test fails once real data exists. Run order must be schema → smoke test → seed, or run the smoke test on a scratch database.
- No product duration. The current MyAlpha UI shows duration.sessionCount and duration.time. sessionCount is derivable (count of episode items). duration.time is the course calendar length — 4,838,400 s is 56 days for a 10-session course, not playback — and nothing in the schema holds or derives it.
- One URL per asset, but real assets have rendition sets. Each production image carries seven or more renditions and each video four download renditions (2160/1080/480/360) with sizes. asset has a single download_url plus one file_size_bytes. Either an asset_rendition child table or a documented CDN-derives-sizes convention is needed.
- No home for presenters or contributors. The UI lists eight hosts and four production credits with photos and role titles. These are named real individuals — personal data. There is no table, and the example seed deliberately contains none. If a credits panel is wanted, that is a data-protection decision (lawful basis, retention, image rights) before it is a schema one. Do not seed real names into a demo database.

- No home for the "what's included" block, intro copy, or `getStarted`. These are presentation content the consumer expects. `getStarted` was the POC stretch goal and was not built. Probably belongs to `product_family`, not `product`.
- `product` has no sequence. MyAlpha orders products with `menu_order`; only `product_family` has a sequence column here, so editions inside a family have no defined order.
- Language codes are messier than two-letter ISO-639-1. Real data holds `de_CH`, `tl_PH`, `pt_BR`, `zh_CN`, `zh_TW` alongside ISO-639-3 codes (`kha`, `zom`, `tcz`, `mni`, `nmf`, `grt`, `ctd`, `iba`), and the reference consumer uses BCP-47 `es-419`. `language.code` is text so all load, but nothing downstream may assume a length or a separator, and no normalisation rule is defined yet.
- "Alpha Youth Series" is a family of nine editions, not one product. Confirmed against MyAlpha: nine primary-language product posts plus roughly 65 translations. The family/product split in §2.7 holds; the seed and any import must be built around it rather than around a single slug.
- Product translations are not products. A translated product post is the same product in another language, and this schema models language on the asset. That is a deliberate choice (D-DOM-01) and the most consequential one for any importer: if a consumer ever needs series listed separately per language, the schema needs revisiting, not the import.
- Extracting from MyAlpha is harder than a query. MyAlpha keeps episode lists in a PHP-serialised `bucket_lists` postmeta and videos in a PHP-serialised `videos` postmeta keyed by provider. MySQL has no `unserialize()`, so a pure-SQL extraction is string surgery and not a general parser. A real import needs a PHP-side export step. Per-language subtitle maps are effectively unreachable from SQL.

4. FUTURE AND PRODUCTION

Estimating basis and confidence: day figures below are dev-days of one engineer, not calendar days, and they assume the same AI-assisted working style as the POC. They are ranges because several depend on decisions nobody has made yet; each carries a confidence label, and the assumptions they rest on are listed in §4.7. Cost figures are indicative list prices — they have not been quoted. Treat the whole section as a planning input requiring review, not a commitment.

4.1 ROADMAP

Four phases. The ordering is not preference: Phase 0 and Phase 1 are prerequisites for anything else being safe, and Phase 2 sits largely in another team's backlog.

Phase	Theme	Why it is here	Blocks
0	Make the current build deployable and survivable	The POC has no migration path, no pipeline, one environment, and a plaintext secret (§3.6). Nothing else should be built on that.	Everything
1	Data foundation: schema gaps and a proper baseline product seed	The catalog cannot be trusted as a source until the baseline products are really in it and the schema can hold what the UI already shows.	Phase 3, and any real pilot
2	RBAC in the Identity Provider, replacing the allow-list stub	Portal authorisation is a configured allow-list, not roles (§3.3, T-02, T-08). Real staff access needs real roles.	Multi-team or multi-country editing
3	WhatsApp Content Distribution onto the API and its new contracts	The motivating consumer. Proves the product removes a repo deploy from a content change.	—

Phases 0 and 1 can overlap once migrations exist. Phase 2 should start early as a conversation even though the code lands later, because the role model is the long-lead item. Phase 3 depends on Phase 1 for content that the schema does not yet hold.

4.2 PRODUCT & TECHNICAL VISION

The POC deliberately built a concept catalog — everything stored, filtered by query parameters — rather than a consumer-shaped payload. The vision keeps that shape and adds the things that make it a system of record rather than a demo:

- One catalog, many consumers. WhatsApp Content Distribution, the Guest App, a modernised Courses service and MyAlpha itself read the same product graph. Each consumer shapes its own payload in its own BFF; the catalog does not grow a per-consumer contract.
- Editing moves to the portal, and content changes stop being deploys. This is the whole proposition. It only becomes true once the baseline data is authored here rather than imported once and diverging — see the fork in §4.3.
- Language stays on the asset, not the product (D-DOM-01). Dubs and subtitles are asset rows; a translated product is not a new product. This is the decision most likely to be challenged by a consumer, and the one most expensive to reverse.
- Country stays an ACL in code, markets stay data. Optional filters never become stored variant rows (D-DOM-02). The reference consumer's 11 pre-computed courseType:audience:country:language variants are precisely the combinatorial explosion this avoids.
- Media stays external. The catalog holds provider, provider asset id, language and URLs. It does not become a media platform, a packager or a player.
- Identity stays in the IDP. No user rows, ever. Roles arrive as claims (Phase 2); the catalog schema does not change to accommodate them, which is why the allow-list stub was acceptable as a stop-gap.

4.3 REMAINING WORK

4.3.1 Production Readiness of What Already Exists (Phase 0)

Each line traces to a limitation in §3.6.

Item	Days	Confidence	Note
Versioned migrations replacing DROP TABLE-and-recreate	3–5	Medium-high	Pick one tool (EF Core migrations, DbUp or Flyway) and back-fill an initial baseline. Hard prerequisite for everything else.
CI/CD pipeline: build both artefacts, merge SPA into wwwroot, deploy to slot, smoke-test, swap	4–6	Medium-high	Removes the manual SPA merge, which §2.5 already calls the weakest part of the setup.
Staging environment, deployment slot, per-environment config	2–3	Medium-high	Needs an App Service plan that supports slots (Standard+).
Bff__ClientSecret to Key Vault with managed identity	1–2	High	Small and unambiguous.
ASPNETCORE_ENVIRONMENT=Production; move UseForwardedHeaders out of the Development branch	0.5–1	High	Currently the app runs as Development in order to get forwarded headers.
Azure Monitor alert rules on the custom events already emitted (§3.2)	1–2	Medium	The events exist; only the rules are missing.
Portal code-splitting (~1.4 MB single chunk) and the responsive pass (FR-POR-09)	2–3	Medium	Deferred honestly in the POC.
Cut over to the production App Service plan and database tier; sanity-load test	2–3	Medium	Plans are already provisioned (§4.5).
Subtotal	16–25		

4.3.2 Data Foundation and a Proper Baseline Product Seed (Phase 1)

The example seed shipped with the POC is three synthetic courses on .invalid URLs. A proper baseline seed means the real global product line present, correct, and reconciled.

The fork that has to be decided first — import once, or author here?

Option	What it means	Cost	Risk
A — one-off import, then author in the portal	Migrate the baseline from MyAlpha once; the portal	Importer is throwaway	Needs content-owner buy-in to change where they work

Option	What it means	Cost	Risk
	becomes the source of truth from that day		
B — repeated import, MyAlpha stays source of truth	The catalog is a read replica of MyAlpha	Importer becomes a maintained sync service, +5–8 days and ongoing	Undermines the product's whole proposition: content changes still require a MyAlpha edit

Recommendation: Option A. Option B keeps the problem the product exists to solve. This is a product decision, not a technical one, and it is the single most important open question in §4.

Item	Days	Confidence	Note
Schema additions: asset renditions, product duration/session count, family content block (gettingStarted/intro/"what's included"), product.sequence	4–6	Medium	All six gaps are itemised in §3.6. Needs migrations (Phase 0) first.
MyAlpha export tool: PHP-side unserialize of bucket_lists / videos to JSON	2–3	Medium	A pure-SQL extraction was prototyped and works, but is string surgery on PHP-serialised data and cannot reach per-language subtitle maps. A PHP export step is the honest approach.
Importer plus mapping and normalisation: language codes, audience vocabulary, markets and the country ACL	3–4	Medium-low	Normalisation rules do not exist yet (§3.6, §4.7).
Media and rendition strategy: which URLs the catalog stores, signed or not	1–2	Low	Depends on a media-hosting decision outside this service.
Reconciliation report and acceptance tests: per-product counts against MyAlpha	2–3	Medium	Without this, "the seed is correct" is an assertion.
Content-owner review and correction cycles	1–2	Low	Calendar-bound, not effort-bound.
Subtotal	13–20		Add 5–8 if Option B is chosen.

4.3.3 RBAC in the Identity Provider (Phase 2)

Today: IDP has role scaffolding but no roles assigned and no role claims on tokens (§1.7), so Product.Bff authorises with a configured allow-list of accounts. That is documented as a stub (T-08) and is not production authorisation. This is work in another team's service, which drives both the range and the low confidence. The cost is dominated by one unanswered question: are roles flat, or scoped by market? Alpha is multi-country, and "editor for Alpha Kenya but not Alpha UK" is a plausible requirement that roughly doubles the work.

Item	Days (flat)	Days (market-scoped)	Confidence
Define and sign off the role model	2–4	3–5	Low — needs product and IDP owners in a room
IDP: role storage, assignment, claims emission, token shape change	6–10	14–22	Low
Migrate and regression-test existing IDP clients against the new token shape	2–4	3–5	Low — depends how many clients exist
Replace the allow-list stub in Product.Bff with policy-based authorisation, plus tests	2–3	3–4	Medium-high — our side is small
Role-aware portal UI (hide what a role cannot do)	1–2	2–3	Medium
Subtotal	13–23	25–39	

Our own share is only 3–5 days. Everything else is IDP-team capacity and cross-team scheduling. Start the role-model conversation in Phase 0 even though the code lands later.

4.3.4 WhatsApp Content Distribution onto the API (Phase 3)

The reference consumer is a static site that client-fetches a pre-computed manifest of 11 variants keyed `courseType:audience:country:language`. Inspecting it against our contract found mismatches that are real work, not mapping trivia:

Mismatch	Detail	Where it lands
It cannot hold a secret	A static site has nowhere to put client credentials (T-01). It needs either build-time generation or its own confidential BFF.	Architecture choice below
Audience vocabulary differs	Consumer uses adult, teenagers, youth; our tags are adults, youth, students	Tag data or a consumer-side map
Locale format differs	Consumer uses BCP-47 es-419; our data holds pt_BR, zh_CN and ISO-639-3 codes	The normalisation rule that §3.6 says does not exist
Episode identity differs	Consumer keys episodes on MyAlpha post ids (e.g. "9320"); we expose UUIDs and (product, code)	A stable public identifier decision
Grouping model	Consumer has groups[].kind (flat and others); we have product_item.grouping	Mapping, probably lossy in one direction
meta shows "11 weeks"	Calendar duration — the missing product-duration concept (§3.6)	Phase 1 schema work
gettingStarted, intro, training videos	Copy blocks with POEditor i18n keys, not catalog data	Phase 1 schema work + an i18n strategy

Architecture options:

Option	How	Days	Confidence	Trade-off
A — build-time manifest generation (recommended)	A pipeline job reads the API with client credentials and emits the manifest; site stays static, no runtime auth, no new deployable	12–19	Medium	Content changes need a rebuild. Mitigate with a scheduled or webhook-triggered rebuild — still far better than a code deploy.
B — runtime confidential BFF	A small server-side component proxies the API; content is live	+6–10 on top of A	Low	A new deployable to host, secure and monitor, for a site that is currently a static artefact.

Option A breakdown: manifest generator 3–4; contract mapping and locale/vocabulary normalisation 2–3; content-block coverage 3–5 (depends on Phase 1); i18n key reconciliation with POEditor 2–4; site changes, acceptance tests and pipeline wiring 2–3.

Recommendation: A first. It proves the value — a content edit in the portal reaching the consumer without a code change — for roughly two-thirds of the cost, and B remains available later without rework of the mapping layer.

4.3.5 Explicitly Still Deferred

Getting Started as a first-class feature; first-party playback or signed URL refresh; live Brightcove/Vimeo integration; caption and VTT handling; MyAlpha country-site shadow overrides; Mixpanel or any analytics store; multi-NAO tenancy; CMS workflow and content approval; translation memory. All were out of scope in §1.7 and remain out.

4.4 TIME, EFFORT, PEOPLE & ROLES

Effort Roll-up

Phase	Dev-days	Of which our team	Confidence
0 — Production readiness	16–25	16–25	Medium-high
1 — Data foundation and baseline seed	13–20	13–20	Medium
2 — RBAC in IDP	13–23	3–5	Low

Phase	Dev-days	Of which our team	Confidence
3 — WhatsApp Content Distribution on the API	12–19	12–19	Medium
Total (recommended path)	54–87	44–69	

The recommended path is Option A on both forks: a one-off import (§4.3.2) and build-time manifest generation (§4.3.4), with flat rather than market-scoped roles. The wider scenario — market-scoped RBAC, a maintained MyAlpha sync, and a runtime consumer BFF — reaches roughly 120 dev-days. The decisions in §4.7 move this estimate more than the engineering does.

Calendar

At two engineers and about four productive days each per week (≈ 8 dev-days/week):

- First production-capable release (Phases 0 + 1, real data on real infrastructure, still allow-list authorisation): 29–45 dev-days $\approx$ 4–6 weeks.
- Our team's full share (44–69 dev-days): $\approx$ 6–9 weeks of development.
- Full scope including RBAC and the consumer: 4–6 calendar months. The gap between 9 weeks of work and 6 months of calendar is not padding — it is cross-team scheduling for Phase 2, content-owner review cycles in Phase 1, and the sign-off latency on the §4.7 decisions. None of those compress by adding engineers.

People & Roles

Role	Phases	Commitment	What they own
Backend engineer (C#)	0, 1, 2, 3	Full	ApplicationRoot, Domain, ProductStore, migrations, importer, manifest generator
Frontend engineer (React/TS)	0, 2	$\sim 40\%$	Portal code-splitting, responsive pass, role-aware UI
IDP engineer (another team)	2	10–18 days	Role model, storage, assignment, claims emission, client migration
Platform / DevOps	0	~ 8 days	Pipeline, slots, Key Vault, managed identity, App Service and database tiers, alert rules
Content owner / data steward	1	Review cycles	Signs off that the baseline catalog is correct. Owns the Option A/B decision.
Product owner	0–3	Ongoing	The §4.7 decisions, consumer priority, role model requirements
QA	1, 3	~ 5 days	Reconciliation acceptance, consumer contract tests
Data protection / DPO	1 (conditional)	Advisory	Only if presenters/contributors are ever modelled (§3.6)

Smallest viable team: two engineers plus part-time platform and IDP support. One engineer could deliver Phases 0 and 1 alone in 29–45 days; Phase 3 needs the manifest and site work, which is a different skill set, and Phase 2 cannot be done by us at all.

4.5 COSTS

These are indicative list prices from general knowledge, not quotes. They are ex-VAT, exclude any EA/CSP discount, and Azure pricing changes. Verify every line in the Azure Pricing Calculator before it reaches a budget. Confidence in the exact figures is low; confidence in the shape — the database dominates, compute may be nearly free — is high.

What We Pay Today

One App Service on F1 Free and one PostgreSQL Flexible Server (v18, Burstable B2s), uksouth, Development subscription. Compute is free; the run cost is the database only, indicatively £40–60/month including storage. Application Insights sits inside the free ingestion allowance at POC volume. This tier is the direct cause of three limitations in §3.6: F1 cannot enable Always On (cold-starts after idle); F1 supports no deployment slots (no safe deploy); and F1 supports no VNet integration (database on public access with firewall rules rather than behind a private endpoint).

What Production Needs, and Why

Line	Change	Indicative £/month	Driver
App Service plan	F1 Free → Standard (S1) or Premium (P1v3)	£0 if absorbed, else ~£55–70 (S1) or ~£95–130 (P1v3)	Always On, deployment slots, autoscale, VNet integration
PostgreSQL	Burstable B2s → General Purpose (e.g. D2ds_v5)	~£120–170	Predictable performance; burstable credits are not a production characteristic
PostgreSQL HA	Add zone-redundant high availability	~doubles the compute line, +£120–170	Survives a zone loss; this is the single largest new cost
Storage + backups	128 GB + retention beyond default	~£15–30	—
Key Vault	New	~£1–3	Removes the plaintext Bff__ClientSecret
App Insights ingestion	Beyond the free allowance	~£0–15	Scales with traffic and log volume
Private endpoint / VNet	New, if the database leaves public access	~£6–12	Closes the public-access limitation
Front Door	Only if the portal separates from the single host	~£25+ traffic	Not needed on the current single-host design (\$2.12)
Production total		~£260–400 (or ~£320–470 if compute not absorbed)	
Staging environment	Burstable database, slot on the shared plan	~£15–50	No staging exists today (\$3.6)

The App Service line is the good news and needs confirming. Better plans are already provisioned, so if this app shares an existing plan the marginal compute cost is close to zero. Worth checking whether the existing plan has headroom, and whether sharing it is acceptable — a noisy neighbour on a shared plan is a real operational coupling, and slots consume plan capacity too.

The honest summary: production is roughly £260–400/month against £40–60 today — about a six- to seven-fold increase at the midpoints (four- to ten-fold across the full range) — and almost all of it is the database: around 90–95% of the total if the App Service plan is shared, about 80% if it is not. Roughly half the database line is high availability alone. If HA is not required for a first production release, the figure drops to roughly £140–230/month. That is a genuine decision to put in front of someone, not a technical default.

Not included: media hosting and CDN egress (media is external to this service by design, but somebody pays for it), non-production environments beyond one staging, and engineering time.

4.6 DEPENDENCIES

Dependency	Needed for	Nature	If it does not arrive
IDP team capacity and a signed-off role model	Phase 2	Blocking, external, long-lead	Portal authorisation stays an allow-list stub. Usable for a small named team; not for multi-country editing.
Platform / subscription owner	Phase 0	Blocking	No production subscription, plan, database tier, Key Vault or private endpoint. Phase 0 cannot complete.
MyAlpha read access (read replica)	Phase 1	Blocking for import	No baseline import; the catalog would have to be authored from scratch in the portal.
Content owners	Phase 1	Blocking for sign-off	The baseline seed cannot be declared correct.

Dependency	Needed for	Nature	If it does not arrive
Media hosting decision (Brightcove / Vimeo / blob + CDN, signed or open)	Phase 1	Blocking for asset URL policy	Asset URLs remain placeholders and the rendition model stays undecided.
POEditor	Phase 3	Blocking if consumer keeps key-based copy	The consumer's gettingStarted copy cannot be resolved.
Data protection / DPO advice	Phase 1, conditional	Advisory	Presenters and contributors stay unmodelled — the safe default.
WhatsApp Content Distribution team	Phase 3	Collaborative	No consumer migration; the product's value stays unproven in production.

One dependency deserves separate mention: credential hygiene. A production MyAlpha read-replica password was found in plaintext in a working directory during this exercise. It was .gitignored and therefore not committed, so this is not an incident on the record — but any Phase 1 import must be given a purpose-scoped, read-only credential issued through the normal route, not a copy of an existing one.

4.7 ASSUMPTIONS

Each of these is currently an assumption, not a decision. The two marked CRITICAL move the estimate in §4.4 more than any engineering choice.

#	Assumption	Risk if wrong
1	CRITICAL — The portal becomes the source of truth for baseline products (Option A, §4.3.2)	If MyAlpha stays authoritative, the importer becomes a maintained sync service (+5–8 days and ongoing cost) and the product no longer removes a deploy from a content change — which was the entire proposition.
2	CRITICAL — Roles are flat, not scoped by market (§4.3.3)	Market-scoped roles take Phase 2 from 13–23 to 25–39 days and materially change the IDP token model.
3	Language stays modelled on the asset, not the product (D-DOM-01)	A consumer needing per-language product listings forces a schema change at the centre of the model, and a re-import.
4	The country–market ACL can stay code rather than data	Country-specific behaviour becomes a deploy; the ACL is currently untested as a unit and undocumented as a table.
5	A locale normalisation rule can be agreed (es-419 / pt_BR / ISO-639-3 all coexist)	Language matching silently mis-resolves and the fallback path returns the wrong asset — a correctness bug that looks like missing content.
6	Build-time manifest generation is acceptable to the consumer (Option A, §4.3.4)	A runtime confidential BFF adds 6–10 days and a new deployable to host, secure and monitor.
7	Media stays external and the catalog stores URLs, not files	The service acquires storage, egress cost and a signed-URL refresh concern it was explicitly scoped away from.
8	Presenters and contributors are not required	Modelling named individuals with photos needs a lawful basis, retention rule and image-rights position before any schema work.
9	A production App Service plan can be shared	Compute goes from roughly £0 to £55–130/month, and a noisy neighbour becomes an operational risk.
10	Zone-redundant database HA is required at first production release	If it is not, run cost roughly halves (§4.5). If it is assumed and skipped, a zone loss takes the catalog offline.
11	One staging environment is enough	Any further environment multiplies the database line, which is the dominant cost.

#	Assumption	Risk if wrong
12	Traffic stays modest — internal consumers, not public anonymous read	Sizing, caching, rate limiting and the Front Door question all reopen.
13	The demo's synthetic three-course seed is never mistaken for real data	.invalid URLs and invented video ids in a production database would be visible to end users.

Not an assumption — a stated risk. The allow-list stub (T-08) is the most likely thing to be mistaken for production readiness, because it works. It authorises correctly for a small, named, carefully-configured group. It has no roles, no delegation, no audit trail of who granted access, and no answer for country-scoped editing. It should be treated as a countdown to Phase 2, not a solution.